

Arduino code for load cells implementation

Generated by Doxygen 1.9.7

Chapter 1

LoadCell and LoadCellController libraries for Arduino

Welcome to the documentation of [LoadCell](#) and [LoadCellController](#) libraries. This documentation provides complete documentation of the libraries.

Chapter 2

Drift correction sketch

This Arduino sketch implements the drift correction strategy that consists of taking weight measurements when the scale is empty as the new reference/offset. This strategy is detailed in [Lab notes](#) at the end of the document.

Chapter 3

Todo List

Member [LoadCellController::easy_start_with_questions](#) (byte loadcell_num, byte dout, byte pd_sck, byte gain=128)

This could be done in the future to simplify the start-up of a [LoadCell](#) especially for non-programmers.

Chapter 4

Module Index

4.1 Modules

Here is a list of all modules:

LoadCell functions for LoadCellController ??

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

HX711	
LoadCell	??
LoadCellController	??

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

LoadCell	This library extends the functionality of the HX711 to make it compatible with the LoadCellController library	??
LoadCellController	This library allows to control and manage many load cells with a LoadCellController	??

Chapter 7

File Index

7.1 File List

Here is a list of all documented files with brief descriptions:

LoadCellControllerlibrary / LoadCellController.h	??
LoadCellLibrary/ LoadCell.h	??

Chapter 8

Module Documentation

8.1 LoadCell functions for LoadCellController

Functions that are the member functions of the [LoadCell](#) class for the [LoadCellController](#) class. They are duplicated in the [LoadCellController](#) class to access the [loadcells](#) through the [LoadCellController](#) instance.

Functions

- float [LoadCellController::get_offset](#) (byte loadcell_num)
Get the offset value for a given [LoadCell](#).
- void [LoadCellController::set_offset](#) (byte loadcell_num, float offset)
Set the offset value for a given [LoadCell](#).
- float [LoadCellController::get_scale](#) (byte loadcell_num)
Get the scale coefficient value for a given [LoadCell](#).
- void [LoadCellController::set_scale](#) (byte loadcell_num, float scale)
Set the scale coefficient value for a given [LoadCell](#).
- void [LoadCellController::set_tare_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for calibration of offset for a given [LoadCell](#).
- byte [LoadCellController::get_tare_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for calibration of offset for a given [LoadCell](#).
- void [LoadCellController::set_scale_coeff_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#).
- byte [LoadCellController::get_scale_coeff_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for calibration of scale calibration for a given [LoadCell](#).
- void [LoadCellController::set_weight_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for weight measurements for a given [LoadCell](#).
- byte [LoadCellController::get_weight_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for measurements of a given [LoadCell](#)'s weight.
- void [LoadCellController::set_all_loadcells_tare_n_readings](#) (int n_readings)
Set the number of readings averaged for calibration of offset for all [LoadCells](#).
- void [LoadCellController::set_all_loadcells_scale_coeff_n_readings](#) (int n_readings)
Set the number of readings averaged for calibration of scale coefficient for all [LoadCells](#).
- void [LoadCellController::set_all_loadcells_weight_n_readings](#) (int n_readings)
Set the number of readings averaged for weight measurements for all [LoadCells](#).
- long [LoadCellController::read_raw_average](#) (byte loadcell_num)

- Read the output of the [LoadCell](#) and average weight_n_readings readings.*
 - long [LoadCellController::read_tare_average](#) (byte loadcell_num)
- Read the output of the [LoadCell](#) and average tare_n_readings readings.*
 - long [LoadCellController::read_scale_coeff_average](#) (byte loadcell_num)
- Read the output of the [LoadCell](#) and average scale_coeff_n_readings readings.*
 - double [LoadCellController::get_raw_value](#) (byte loadcell_num)
- Read the raw output of the [LoadCell](#) and subtracts the offset.*
 - float [LoadCellController::get_weight](#) (byte loadcell_num)
- Read the weight of the object currently on the [LoadCell](#).*
 - void [LoadCellController::tare](#) (byte loadcell_num)
- Read the raw output of the [LoadCell](#) and set this value to the offset.*
 - void [LoadCellController::begin](#) (byte loadcell_num, byte dout, byte pd_sck, byte gain=128)
- Begin the communication with the HX711 for a given [LoadCell](#).*
 - bool [LoadCellController::is_ready](#) (byte loadcell_num)
- Verify if the HX711 is ready for a reading.*
 - void [LoadCellController::wait_ready](#) (byte loadcell_num, unsigned long delay_ms=0)
- Suspend the execution of the code until the HX711 becomes ready.*
 - bool [LoadCellController::wait_ready_retry](#) (byte loadcell_num, int retries=3, unsigned long delay_ms=0)
- Suspend the execution of the code until the HX711 becomes ready or for a given number of retries.*
 - bool [LoadCellController::wait_ready_timeout](#) (byte loadcell_num, unsigned long timeout=1000, unsigned long delay_ms=0)
- Suspend the execution of the code until the HX711 becomes ready or for a given timeout.*
 - void [LoadCellController::power_up](#) (byte loadcell_num)
- Wake up the chip after power down mode.*
 - void [LoadCellController::power_down](#) (byte loadcell_num)
- Wake up the chip after power down mode.*

8.1.1 Detailed Description

Functions that are the member functions of the [LoadCell](#) class for the [LoadCellController](#) class. They are duplicated in the [LoadCellController](#) class to access the [loadcells](#) through the [LoadCellController](#) instance.

Note

[LoadCell](#) class member functions can also be accessed through the [LoadCell](#) instance. The two possibilities are available.

8.1.2 Function Documentation

8.1.2.1 begin()

```
void LoadCellController::begin (
    byte loadcell_num,
    byte dout,
    byte pd_sck,
    byte gain = 128 )
```

Begin the communication with the HX711 for a given [LoadCell](#).

This function begins the communication with the HX711 that is performing a voltage reading of the Wheatstone bridge on the load cell to convert it in a 24-bits digital raw output. This function needs to be called before performing measurements of weight.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>dout</i>	Digital pin (or analog) connected to the DOUT output pin of the HX711.
<i>pd_sck</i>	Digital pin (or analog) connected to the SCK output of the HX711.
<i>gain</i>	Gain of the HX711. Default is 128.

The `gain` argument can take the following values: 128, 64 and 32. For a gain of 128 and 64, channel A of the HX711 should be used. For a gain of 32, channel B should be used.

8.1.2.2 `get_offset()`

```
float LoadCellController::get_offset (
    byte loadcell_num )
```

Get the offset value for a given [LoadCell](#).

This function gets the offset member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Offset value.

8.1.2.3 `get_raw_value()`

```
double LoadCellController::get_raw_value (
    byte loadcell_num )
```

Read the raw output of the [LoadCell](#) and subtracts the offset.

This function reads the average of readings of the [LoadCell](#) raw output with `read_raw_average()` then subtracts the offset accessed with `get_offset()`.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The raw averaged reading of the [LoadCell](#) difference with the offset.

8.1.2.4 `get_scale()`

```
float LoadCellController::get_scale (
    byte loadcell_num )
```

Get the scale coefficient value for a given [LoadCell](#).

This function gets the scale coefficient member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Scale coefficient value.

8.1.2.5 `get_scale_coeff_n_readings()`

```
byte LoadCellController::get_scale_coeff_n_readings (
    byte loadcell_num )
```

Get the number of readings averaged for calibration of scale calibration for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of offset for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Number of readings averaged for scale coefficient calibration.

8.1.2.6 `get_tare_n_readings()`

```
byte LoadCellController::get_tare_n_readings (
    byte loadcell_num )
```

Get the number of readings averaged for calibration of offset for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of offset for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Number of readings averaged for offset calibration.

8.1.2.7 get_weight()

```
float LoadCellController::get_weight (
    byte loadcell_num )
```

Read the weight of the object currently on the [LoadCell](#).

This function reads the average of readings of the [LoadCell](#) raw output with [read_raw_average\(\)](#) before converting this average value to a mass value using the calibration function -> $\text{output} = \text{scale_coeff} * \text{mass} + \text{offset}$.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

The averaged mass of the object on the [LoadCell](#).

8.1.2.8 get_weight_n_readings()

```
byte LoadCellController::get_weight_n_readings (
    byte loadcell_num )
```

Get the number of readings averaged for measurements of a given [LoadCell](#)'s weight.

This function retrieves the number of readings that are averaged for measurements of a given [LoadCell](#)'s weight.

Parameters

<code>loadcell_num</code>	<code>loadcell_num</code> Number of the LoadCell .
---------------------------	--

Returns

Number of reading averaged for a weight measurements.

8.1.2.9 is_ready()

```
bool LoadCellController::is_ready (
    byte loadcell_num )
```

Verify if the HX711 is ready for a reading.

This function verifies if the HX711 is ready for retrieval the next reading. According to the datasheet of the HX711, when output data is not ready for retrieval, digital output pin DOUT is high. Serial clock input PD_SCK should be low. When DOUT goes to low, it indicates data is ready for retrieval. The latter condition is verified in this member function.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

Boolean showing if the HX711 is ready for retrieval.

8.1.2.10 power_down()

```
void LoadCellController::power_down (
    byte loadcell_num )
```

Wake up the chip after power down mode.

This function powers down the HX711 after calling [power_up\(\)](#) member function.

Note

This function doesn't stop the alimentation of the Wheatstone on the load cell, only suspend the HX711.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

8.1.2.11 power_up()

```
void LoadCellController::power_up (
    byte loadcell_num )
```

Wake up the chip after power down mode.

This function powers up the HX711 after calling [power_down\(\)](#) member function.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

8.1.2.12 read_raw_average()

```
long LoadCellController::read_raw_average (
    byte loadcell_num )
```

Read the output of the [LoadCell](#) and average `weight_n_readings` readings.

This function does `weight_n_readings` readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The raw averaged reading of the [LoadCell](#).

8.1.2.13 read_scale_coeff_average()

```
long LoadCellController::read_scale_coeff_average (
    byte loadcell_num )
```

Read the output of the [LoadCell](#) and average scale_coeff_n_readings readings.

This function does scale_coeff_n_readings readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The raw averaged reading of the [LoadCell](#) for the scale coefficient calibration.

8.1.2.14 read_tare_average()

```
long LoadCellController::read_tare_average (
    byte loadcell_num )
```

Read the output of the [LoadCell](#) and average tare_n_readings readings.

This function does tare_n_readings readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The raw averaged reading of the [LoadCell](#) for the tare offset calibration.

8.1.2.15 set_all_loadcells_scale_coeff_n_readings()

```
void LoadCellController::set_all_loadcells_scale_coeff_n_readings (
    int n_readings )
```

Set the number of readings averaged for calibration of scale coefficient for all LoadCells.

This function sets the number of readings to be averaged for calibration of scale coefficient for all LoadCells.

Parameters

<i>n_readings</i>	Number of readings averaged for calibration of scale coefficient.
-------------------	---

8.1.2.16 set_all_loadcells_tare_n_readings()

```
void LoadCellController::set_all_loadcells_tare_n_readings (
    int n_readings )
```

Set the number of readings averaged for calibration of offset for all LoadCells.

This function sets the number of readings to be averaged for calibration of offset for all LoadCells.

Parameters

<i>n_readings</i>	Number of readings averaged for calibration of offset.
-------------------	--

8.1.2.17 set_all_loadcells_weight_n_readings()

```
void LoadCellController::set_all_loadcells_weight_n_readings (
    int n_readings )
```

Set the number of readings averaged for weight measurements for all LoadCells.

This function sets the number of readings to be averaged for weight measurements for all LoadCells.

Parameters

<i>n_readings</i>	Number of readings averaged for weight measurements.
-------------------	--

8.1.2.18 set_offset()

```
void LoadCellController::set_offset (
    byte loadcell_num,
    float offset )
```

Set the offset value for a given [LoadCell](#).

This function sets the offset member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>offset</i>	Offset value.

8.1.2.19 set_scale()

```
void LoadCellController::set_scale (
    byte loadcell_num,
    float scale )
```

Set the scale coefficient value for a given [LoadCell](#).

This function sets the scale coefficient member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>scale</i>	Scale coefficient value.

8.1.2.20 set_scale_coeff_n_readings()

```
void LoadCellController::set_scale_coeff_n_readings (
    byte loadcell_num,
    int n_readings )
```

Set the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>n_readings</i>	Number of readings.

8.1.2.21 set_tare_n_readings()

```
void LoadCellController::set_tare_n_readings (
    byte loadcell_num,
    int n_readings )
```

Set the number of readings averaged for calibration of offset for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of offset for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>n_readings</i>	Number of readings.

8.1.2.22 set_weight_n_readings()

```
void LoadCellController::set_weight_n_readings (
    byte loadcell_num,
    int n_readings )
```

Set the number of readings averaged for weight measurements for a given [LoadCell](#).

This function sets the number of readings to be averaged for weight measurements for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>n_readings</i>	Number of reading averaged for a weight measurements.

8.1.2.23 tare()

```
void LoadCellController::tare (
    byte loadcell_num )
```

Read the raw output of the [LoadCell](#) and set this value to the offset.

This function does tare_n_reading readings to get the average for the offset. It then sets this value to the member variable OFFSET.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

8.1.2.24 wait_ready()

```
void LoadCellController::wait_ready (
    byte loadcell_num,
    unsigned long delay_ms = 0 )
```

Suspend the execution of the code until the HX711 becomes ready.

This function suspends the execution of the code with a while loop relying on the [is_ready\(\)](#) condition. When called, this function will stop temporarily the code execution until the HX711 becomes ready.

Warning

Using this function can completely stop the code execution if a technical problem happens with a HX711. It is safer to handle the retrieval of a reading with [wait_ready_retry\(\)](#) or [wait_ready_timeout\(\)](#)

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

8.1.2.25 wait_ready_retry()

```
bool LoadCellController::wait_ready_retry (
    byte loadcell_num,
    int retries = 3,
    unsigned long delay_ms = 0 )
```

Suspend the execution of the code until the HX711 becomes ready or for a given number of retries.

This function suspends the execution of the code with a while loop relying on the [is_ready\(\)](#) condition. When called, this function will stop temporarily the code execution until the HX711 becomes ready or for a given number of retries. This prevents from a endless suspension of the code.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>retries</i>	Number of retries to verify the "readiness state" of the HX711. Default is 3.
<i>delay_ms</i>	Delay between two verification of the "readiness state" of the HX711. Default is 0.

8.1.2.26 wait_ready_timeout()

```
bool LoadCellController::wait_ready_timeout (
    byte loadcell_num,
    unsigned long timeout = 1000,
    unsigned long delay_ms = 0 )
```

Suspend the execution of the code until the HX711 becomes ready or for a given timeout.

This function suspends the execution of the code with a while loop relying on the [is_ready\(\)](#) condition. When called, this function will stop temporarily the code execution until the HX711 becomes ready or for a given timeout. If after the timeout the HX711 is not ready for retrieval, the code execution will continue. This prevents from a endless suspension of the code.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>timeout</i>	Timeout of the suspension of execution of the code in milliseconds. Default is 1000 ms.
<i>delay_ms</i>	Delay between two verification of the "readiness state" of the HX711 in milliseconds. Default is 0.

Chapter 9

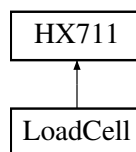
Class Documentation

9.1 LoadCell Class Reference

This library extends the functionality of the HX711 to make it compatible with the [LoadCellController](#) library.

```
#include <LoadCell.h>
```

Inheritance diagram for LoadCell:



Public Member Functions

- **LoadCell ()**
Construct a [LoadCell](#) object.
- long **read_raw_average ()**
Read the output of the [LoadCell](#) and average [weight_n_readings](#) readings.
- long **read_tare_average ()**
Read the output of the [LoadCell](#) and average [tare_n_readings](#) readings.
- long **read_scale_coeff_average ()**
Read the output of the [LoadCell](#) and average [scale_coeff_n_readings](#) readings.
- double **get_raw_value ()**
Read the raw output of the [LoadCell](#) and subtracts the offset.
- float **get_weight ()**
Read the weight of the object currently on the [LoadCell](#).
- void **tare ()**
Read the raw output of the [LoadCell](#) and set this value to the offset.
- void **set_tare_n_readings (int tare_n_readings)**
Set the number of readings averaged for the determination of the tare offset.
- int **get_tare_n_readings ()**
Get the number of readings averaged for the determination of the tare offset.
- void **set_scale_coeff_n_readings (int scale_coeff_n_readings)**

- Set the number of readings averaged for the determination of the scale coefficient.*
 - int `get_scale_coeff_n_readings` ()
- Get the number of readings averaged for the determination of the scale coefficient.*
 - void `set_weight_n_readings` (int `weight_n_readings`)
- Set the number of readings averaged for a weight reading.*
 - int `get_weight_n_readings` ()
- Get the number of readings averaged for a weight reading.*

Protected Attributes

- byte `tare_n_readings` = 50
 - Number of readings averaged to determine the tare offset.*
- byte `scale_coeff_n_readings` = 50
 - Number of readings averaged to determine the scale coefficient.*
- byte `weight_n_readings` = 5
 - Number of readings averaged for a weight reading.*

9.1.1 Detailed Description

This library extends the functionality of the HX711 to make it compatible with the [LoadCellController](#) library.

This Arduino library allows to specify the number of readings averaged for the three possible situations:

- Weight reading
- Output reading for offset calibration
- Output reading for scale coefficient calibration

The encapsulation of these parameters requires the rewriting of a few member functions of the HX711 library. They do the same thing but, with the encapsulation of the three parameters listed above, they don't require any arguments.

Most importantly, the encapsulation of these parameters is useful for the [LoadCellController](#) library, which will control many LoadCells.

HX711 library: <https://github.com/bogde/HX711>

[LoadCellController](#) library: <https://github.com/DCC-Lab/IntelligentCage/tree/master/Loadcell/Arduino%20libraries/LoadCellControllerlibrary>

Author

Nathan Bérubé

Date

August 14, 2023

Version

1.0.0

9.1.2 Member Function Documentation

9.1.2.1 `get_raw_value()`

```
double LoadCell::get_raw_value ( )
```

Read the raw output of the [LoadCell](#) and subtracts the offset.

This function reads the average of readings of the [LoadCell](#) raw output with [read_raw_average\(\)](#) then subtracts the offset accessed with [get_offset\(\)](#).

Returns

The raw averaged reading of the [LoadCell](#) difference with the offset.

9.1.2.2 `get_scale_coeff_n_readings()`

```
int LoadCell::get_scale_coeff_n_readings ( )
```

Get the number of readings averaged for the determination of the scale coefficient.

This function returns the member variable [scale_coeff_n_readings](#).

Returns

The number of readings averaged for the determination of the tare offset.

9.1.2.3 `get_tare_n_readings()`

```
int LoadCell::get_tare_n_readings ( )
```

Get the number of readings averaged for the determination of the tare offset.

This function returns the member variable [tare_n_readings](#).

Returns

The number of readings averaged for the determination of the tare offset.

9.1.2.4 `get_weight()`

```
float LoadCell::get_weight ( )
```

Read the weight of the object currently on the [LoadCell](#).

This function reads the average of readings of the [LoadCell](#) raw output with [read_raw_average\(\)](#) before converting this average of raw output to a mass value using the calibration function -> $\text{output} = \text{scale_coeff} * \text{mass} + \text{offset}$.

Returns

The averaged mass of the object on the [LoadCell](#).

9.1.2.5 get_weight_n_readings()

```
int LoadCell::get_weight_n_readings ( )
```

Get the number of readings averaged for a weight reading.

This function returns the member variable [scale_coeff_n_readings](#).

Returns

The number of readings averaged for the determination of the tare offset.

9.1.2.6 read_raw_average()

```
long LoadCell::read_raw_average ( )
```

Read the output of the [LoadCell](#) and average [weight_n_readings](#) readings.

This function does [weight_n_readings](#) readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output.

Returns

The raw averaged reading of the [LoadCell](#).

9.1.2.7 read_scale_coeff_average()

```
long LoadCell::read_scale_coeff_average ( )
```

Read the output of the [LoadCell](#) and average [scale_coeff_n_readings](#) readings.

This function does [scale_coeff_n_readings](#) readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Returns

The raw averaged reading of the [LoadCell](#) for the scale coefficient calibration.

9.1.2.8 read_tare_average()

```
long LoadCell::read_tare_average ( )
```

Read the output of the [LoadCell](#) and average [tare_n_readings](#) readings.

This function does [tare_n_readings](#) readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Returns

The raw averaged reading of the [LoadCell](#) for the tare offset calibration.

9.1.2.9 set_scale_coeff_n_readings()

```
void LoadCell::set_scale_coeff_n_readings (
    int scale_coeff_n_readings )
```

Set the number of readings averaged for the determination of the scale coefficient.

This function sets the member variable [scale_coeff_n_readings](#). It also limits the range of value that is possible for this member variable -> [scale_coeff_n_readings](#) [1, 255].

Anything below 1 will be set to 1 and anything above 255 will be set to 255 to be able to store in a byte.

Parameters

<code>scale_coeff_n_readings</code>	The number of readings averaged for the determination of the tare offset.
-------------------------------------	---

9.1.2.10 set_tare_n_readings()

```
void LoadCell::set_tare_n_readings (
    int tare_n_readings )
```

Set the number of readings averaged for the determination of the tare offset.

This function sets the member variable `tare_n_readings`. It also limits the range of value that is possible for this member variable -> [tare_n_readings](#) [1, 255].

Anything below 1 will be set to 1 and anything above 255 will be set to 255 to be able to store in a byte.

Parameters

<code>tare_n_readings</code>	The number of readings averaged for the determination of the tare offset.
------------------------------	---

9.1.2.11 set_weight_n_readings()

```
void LoadCell::set_weight_n_readings (
    int weight_n_readings )
```

Set the number of readings averaged for a weight reading.

This function sets the member variable `weight_n_readings`. It also limits the range of value that is possible for this member variable -> [weight_n_readings](#) [1, 255].

Anything below 1 will be set to 1 and anything above 255 will be set to 255 to be able to store in a byte.

Parameters

<code>weight_n_readings</code>	The number of readings averaged for the determination of the tare offset.
--------------------------------	---

9.1.2.12 tare()

```
void LoadCell::tare ( )
```

Read the raw output of the [LoadCell](#) and set this value to the offset.

This function does `tare_n_reading` readings to get the average for the offset. It then sets this value to the member variable `OFFSET` that is private.

The documentation for this class was generated from the following files:

- LoadCellLibrary/[LoadCell.h](#)
- LoadCellLibrary/[LoadCell.cpp](#)

9.2 LoadCellController Class Reference

This library allows to control and manage many load cells with a [LoadCellController](#).

```
#include <LoadCellController.h>
```

Public Member Functions

- **LoadCellController ()**
Construct a [LoadCellController](#) object.
- void **add_loadcell** ([LoadCell](#) &loadcell)
Add a new [LoadCell](#) to the [LoadCellController](#).
- void **easy_start_with_params** (byte loadcell_num, byte dout, byte pd_sck, bool calibrate_offset=false, bool calibrate_scale=false, bool read_offset_persistent_memory=false, bool read_scale_persistent_memory=false, bool save_offset_persistent_memory=false, bool save_scale_persistent_memory=false, float tare_offset=0, float scale_coeff=0, byte gain=128)
Easy function managing the start-up of a [LoadCell](#) with parameters as input.
- void **save_offset_to_persistent_memory** (byte loadcell_num)
Save the offset of a given [LoadCell](#) to persistent memory.
- void **save_scale_coeff_to_persistent_memory** (byte loadcell_num)
Save the scale coefficient of a given [LoadCell](#) to persistent memory.
- long **read_offset_from_persistent_memory** (byte loadcell_num)
Read the offset of a given [LoadCell](#) from persistent memory.
- float **read_scale_coeff_from_persistent_memory** (byte loadcell_num)
Read the scale coefficient of a given [LoadCell](#) to persistent memory.
- void **calibrate_both_params** (byte loadcell_num)
Proceed to the calibration of both parameters and set them for a [LoadCell](#).
- void **calibrate_tare_offset** (byte loadcell_num)
Proceed to the calibration of the offset and set it for a [LoadCell](#).
- void **calibrate_scale_coeff** (byte loadcell_num)
Proceed to the calibration of the scale coefficient and set it for a [LoadCell](#).
- float **determine_offset** (byte loadcell_num)
Proceed to the calibration of the offset and return the offset value.
- float **determine_scale_coeff** (byte loadcell_num)
Proceed to the calibration of the scale coefficient and return the scale coefficient value.
- float **calculate_scale_coeff** (byte loadcell_num, float output, float mass)
- int **get_offset_eeprom_address** (byte loadcell_num)
Get the EEPROM address for the offset of a [LoadCell](#).
- int **get_scale_coeff_eeprom_address** (byte loadcell_num)
Get the EEPROM address for the scale coefficient of a [LoadCell](#).
- String **get_offset_file_name** (byte loadcell_num)
Get the SPIFFS file name for the offset of a [LoadCell](#).
- String **get_scale_coeff_file_name** (byte loadcell_num)
Get the SPIFFS file name for the scale coefficient of a [LoadCell](#).
- float **get_offset** (byte loadcell_num)
Get the offset value for a given [LoadCell](#).
- void **set_offset** (byte loadcell_num, float offset)
Set the offset value for a given [LoadCell](#).
- float **get_scale** (byte loadcell_num)
Get the scale coefficient value for a given [LoadCell](#).

- void [set_scale](#) (byte loadcell_num, float scale)
Set the scale coefficient value for a given [LoadCell](#).
- void [set_tare_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for calibration of offset for a given [LoadCell](#).
- byte [get_tare_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for calibration of offset for a given [LoadCell](#).
- void [set_scale_coeff_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#).
- byte [get_scale_coeff_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for calibration of scale calibration for a given [LoadCell](#).
- void [set_weight_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for weight measurements for a given [LoadCell](#).
- byte [get_weight_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for measurements of a given [LoadCell](#)'s weight.
- void [set_all_loadcells_tare_n_readings](#) (int n_readings)
Set the number of readings averaged for calibration of offset for all [LoadCells](#).
- void [set_all_loadcells_scale_coeff_n_readings](#) (int n_readings)
Set the number of readings averaged for calibration of scale coefficient for all [LoadCells](#).
- void [set_all_loadcells_weight_n_readings](#) (int n_readings)
Set the number of readings averaged for weight measurements for all [LoadCells](#).
- long [read_raw_average](#) (byte loadcell_num)
Read the output of the [LoadCell](#) and average weight_n_readings readings.
- long [read_tare_average](#) (byte loadcell_num)
Read the output of the [LoadCell](#) and average tare_n_readings readings.
- long [read_scale_coeff_average](#) (byte loadcell_num)
Read the output of the [LoadCell](#) and average scale_coeff_n_readings readings.
- double [get_raw_value](#) (byte loadcell_num)
Read the raw output of the [LoadCell](#) and subtracts the offset.
- float [get_weight](#) (byte loadcell_num)
Read the weight of the object currently on the [LoadCell](#).
- void [tare](#) (byte loadcell_num)
Read the raw output of the [LoadCell](#) and set this value to the offset.
- void [begin](#) (byte loadcell_num, byte dout, byte pd_sck, byte gain=128)
Begin the communication with the HX711 for a given [LoadCell](#).
- bool [is_ready](#) (byte loadcell_num)
Verify if the HX711 is ready for a reading.
- void [wait_ready](#) (byte loadcell_num, unsigned long delay_ms=0)
Suspend the execution of the code until the HX711 becomes ready.
- bool [wait_ready_retry](#) (byte loadcell_num, int retries=3, unsigned long delay_ms=0)
Suspend the execution of the code until the HX711 becomes ready or for a given number of retries.
- bool [wait_ready_timeout](#) (byte loadcell_num, unsigned long timeout=1000, unsigned long delay_ms=0)
Suspend the execution of the code until the HX711 becomes ready or for a given timeout.
- void [power_up](#) (byte loadcell_num)
Wake up the chip after power down mode.
- void [power_down](#) (byte loadcell_num)
Wake up the chip after power down mode.
- byte [number_of_loadcells](#) ()
Get the number of [LoadCell](#) of the [LoadCellController](#).
- bool [is_loadcell_num_in_range](#) (byte loadcell_num)
Verify if a [LoadCell](#) number is in the range of the [LoadCellController](#).
- void [clear_eeprom](#) (int start=0, int end=1024)
Clear the EEPROM with 0 value.

Protected Member Functions

- void [easy_calibration_with_params](#) (byte loadcell_num, bool cal_tare, bool cal_scale)
Easy function managing the calibration of a [LoadCell](#).
- void [easy_save_to_persistent_memory_with_params](#) (byte loadcell_num, bool save_offset, bool save_scale)
Easy function managing the saving of calibration parameters to persistent memory.
- void [easy_read_from_persistent_memory_with_params](#) (byte loadcell_num, bool read_offset, bool read_scale)
Easy function managing the reading of calibration from persistent memory.
- void [easy_start_with_questions](#) (byte loadcell_num, byte dout, byte pd_sck, byte gain=128)
Easy function for start with questions through the serial monitor.
- void [easy_handle_exceptions](#) (byte loadcell_num, bool calibrate_offset=false, bool calibrate_scale=false, bool read_offset_persistent_memory=false, bool read_scale_persistent_memory=false, bool save_offset_persistent_memory=false, bool save_scale_persistent_memory=false, float tare_offset=0, float scale_coeff=0)
Easy function used to handle possible conflicts in [easy_start_with_params\(\)](#).

Protected Attributes

- [LoadCell](#) * [loadcells](#) [10]
Array containing the pointer of the [LoadCell](#) controlled by the [LoadCellController](#).
- int [n_loadcell](#) = 0
Member variable used to keep track of the number of LoadCells added.

9.2.1 Detailed Description

This library allows to control and manage many load cells with a [LoadCellController](#).

The [LoadCellController](#) library offers many features:

- Calibrate load cells, both parameters (offset and scale coefficient)
 - Member functions encapsulate the scripts needed for the calibration for both parameters. The interaction with the user is done inside the member functions.
- Save the calibration parameters to persistent memory (tested on Arduino Uno August 2023)
The library can automatically detect if an Arduino Uno (ATmega328p chip) or a FireBeetle (ESP8266) is used. For Arduino Uno, EEPROM is used to store calibration parameters. For FireBeetle, SPIFFS is used.
The calibration parameters can be saved on the EEPROM after calibration for Atmega328p chip (Arduino Uno). These types of persistent memory are useful when it comes to saving because the saved values don't erase themselves when the power supply of the microcontroller is cut. The saved parameters can be reused at another moment. (tested on Arduino Uno and FireBeetle November 2023)
- Read the calibration parameters from persistent memory (tested on Arduino Uno August 2023)
 - The calibration parameters can also be read from persistent memory after being saved.
- "Easy" start-up function that will handle everything related to the calibration and manage the calibration parameters.
 - The [LoadCellController](#) class offers ready-to-use "easy" functions that simplify the start-up of a load cell. Many scenarios are possible for the start-up of a load cell when it comes to calibration and the management of the calibration parameters.

- A simple function `easy_start_with_params()` takes in arguments many parameters to choose the appropriate option. Only a few parameters are needed.
- This is useful for those who don't want to create their own sketch.
- All the member functions that begin with "easy" are for easy start-up and don't implement any other functionalities than the other member functions. They only offer a "all-in-one" option for those who don't want to code.
- The `LoadCellController` class allows to access the load cells through the controller.

The load cells are added with the member function `add_loadcell()`. After that, all load cells are under the control of the `LoadCellController` and the member functions of the `LoadCell` class are accessed through the controller.

This access to the LoadCells through the `LoadCellController` requires to specify to `LoadCellController` a parameter corresponding to the number identifying the `LoadCell` concerned (1, 2, ...). Every member functions that have as input the number of a `LoadCell` (`n_loadcell`) will verify if the `LoadCell` number is in a coherent range according to the number of LoadCells added to the controller according to the member variable `n_loadcell`. If the value is out of the range, the code will be suspended with a endless `while()` loop and display on the serial monitor an error message specifying where the problem occurred (in which function). This small if bloc is placed at the beginning

```
if (is_loadcell_num_in_range(loadcell_num) == false) {
  Serial.println();
  Serial.println();
  Serial.println();
  Serial.print(F("ERROR 1 name_of_the_function(): loadcell number "));
  Serial.print(loadcell_num);
  Serial.println(F(" is out of range."));
  while(1);
}
```

HX711 library: <https://github.com/bogde/HX711>

`LoadCell` library: <https://github.com/DCC-Lab/IntelligentCage/tree/master/↔Loadcell/Arduino%20libraries/LoadCellLibrary>

Author

Nathan Bérubé

Date

November 1st, 2023

Version

2.0.0

9.2.2 Member Function Documentation

9.2.2.1 `add_loadcell()`

```
void LoadCellController::add_loadcell (
    LoadCell & loadcell )
```

Add a new `LoadCell` to the `LoadCellController`.

This function adds a new `LoadCell` to the controller. It will add the pointer of the `LoadCell` object to the next index in the `loadcells` array and increase the value `n_loadcell` by 1 to keep track of the number of `LoadCell`.

The first added `LoadCell` will be load cell 1, the second one will be load cell 2 and so on.

Parameters

<i>loadcell</i>	LoadCell object added to the LoadCellController .
-----------------	---

9.2.2.2 calibrate_both_params()

```
void LoadCellController::calibrate_both_params (
    byte loadcell_num )
```

Proceed to the calibration of both parameters and set them for a [LoadCell](#).

This function begins the calibration of both offset and scale coefficient by calling the member functions [calibrate_offset\(\)](#) and [calibrate_scale_coeff\(\)](#) that interact with the user to determine and set both calibration parameters.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

9.2.2.3 calibrate_scale_coeff()

```
void LoadCellController::calibrate_scale_coeff (
    byte loadcell_num )
```

Proceed to the calibration of the scale coefficient and set it for a [LoadCell](#).

This function use the member function [determine_scale_coeff\(\)](#) to get the scale coefficient from the calibration done by the user. This member function then sets this scale coefficient value to the member variable of the [LoadCell](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

9.2.2.4 calibrate_tare_offset()

```
void LoadCellController::calibrate_tare_offset (
    byte loadcell_num )
```

Proceed to the calibration of the offset and set it for a [LoadCell](#).

This function use the member function [determine_offset\(\)](#) to get the offset from the calibration done by the user. This member function then sets this offset value to the member variable of the [LoadCell](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

9.2.2.5 clear_eeprom()

```
void LoadCellController::clear_eeprom (
    int start = 0,
    int end = 1024 )
```

Clear the EEPROM with 0 value.

This function writes a 0 at all index on the EEPROM for a given range.

Note

The `end` parameters has a default value of 1024 since it is the size of the Arduino Uno EEPROM. It might not be the case with other microcontroller.

Parameters

<i>start</i>	First index to clear. Default is 0
<i>end</i>	Last index to clear. Default is 1024

9.2.2.6 determine_offset()

```
float LoadCellController::determine_offset (
    byte loadcell_num )
```

Proceed to the calibration of the offset and return the offset value.

This function interacts with the user through the serial monitor to get an offset value for a given [LoadCell](#).

This member function asks to the user to remove any object from the load cell and send "t" with the serial monitor when it is done. After that, a output measurement is done with [read_tare_average\(\)](#) to get a raw output value. This value is returned.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The offset value.

9.2.2.7 determine_scale_coeff()

```
float LoadCellController::determine_scale_coeff (
    byte loadcell_num )
```

Proceed to the calibration of the scale coefficient and return the scale coefficient value.

This function interacts with the user through the serial monitor to get a scale coefficient value for a given [LoadCell](#).

This member function asks the user to specify the number of reference weights that will be used to calibrate the load cell.

After that, the user is asked to put the first reference weight on the loadcell and send the weight value through the serial monitor. A weight measurement is done using the member function [read_scale_coeff_average\(\)](#). A couple for our calibration linear function $\rightarrow \text{output} = \text{scale_coeff} * \text{mass} + \text{offset}$ is obtained (known mass, measured output). The scale coefficient can now be calculated.

The process described above can be repeated for each reference weight to get a scale coefficient value.

The average of all scale coefficients is taken to get the finale value returned by the function.

Note

With this simple method, the scale coefficient depends of the offset. Finding both calibration parameters (offset and scale coefficient) at the same time with a curve fit algorithm could be more effective.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The scale coefficient value.

9.2.2.8 easy_calibration_with_params()

```
void LoadCellController::easy_calibration_with_params (
    byte loadcell_num,
    bool cal_tare,
    bool cal_scale ) [protected]
```

Easy function managing the calibration of a [LoadCell](#).

This function is managing the possibilities that come with the calibration of the tare offset and the scale coefficient. It is meant to be used inside the member function [easy_start_with_params\(\)](#).

The calibration process of both parameters (offset and scale coefficient) requires a computer to display information on the serial monitor and also to send value through it for the calibration.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>cal_tare</i>	Boolean telling if the offset should be calibrated.
<i>save_scale</i>	Boolean telling if the scale coefficient should be calibrated.

9.2.2.9 easy_handle_exceptions()

```
void LoadCellController::easy_handle_exceptions (
    byte loadcell_num,
```

```

bool calibrate_offset = false,
bool calibrate_scale = false,
bool read_offset_persistent_memory = false,
bool read_scale_persistent_memory = false,
bool save_offset_persistent_memory = false,
bool save_scale_persistent_memory = false,
float tare_offset = 0,
float scale_coeff = 0 ) [protected]

```

Easy function used to handle possible conflicts in [easy_start_with_params\(\)](#).

This function manages the possible conflicts that can happen between the parameters passed to the [easy_start_with_params\(\)](#) member function. Since the user is specifying the starting parameters, it is probable that some are not possible simultaneously. This member function verifies all possible conflicts. If one is detected, the execution will be stopped with infinite while() loop, since it is not possible to do raise exceptions on Arduino (as I know). It is meant to be used inside the member function [easy_start_with_params\(\)](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>calibrate_offset</i>	Boolean telling if the offset should be calibrated. Default is false.
<i>calibrate_scale</i>	Boolean telling if the scale coefficient should be calibrated. Default is false.
<i>read_offset_persistent_memory</i>	Boolean telling if the offset should be read from persistent_memory. Default is false.
<i>read_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be read from persistent_memory. Default is false.
<i>save_offset_persistent_memory</i>	Boolean telling if the offset should be saved to persistent_memory. Default is false.
<i>save_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be saved to persistent_memory. Default is false.
<i>tare_offset</i>	Value of the tare offset. Default is 0.

The `tare_offset` default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

Parameters

<i>scale_coeff</i>	Value of the scale coefficient. Default is 0.
--------------------	---

The `scale_coeff` default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

9.2.2.10 easy_read_from_persistent_memory_with_params()

```

void LoadCellController::easy_read_from_persistent_memory_with_params (
    byte loadcell_num,
    bool read_offset,
    bool read_scale ) [protected]

```

Easy function managing the reading of calibration from persistent memory.

This function is managing the possibilities that come with the reading from persistent memory of the tare offset and the scale coefficient of a [LoadCell](#). It is meant to be used inside the member function [easy_start_with_params\(\)](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>read_offset</i>	Boolean telling if the offset should be read from persistent memory.
<i>read_scale</i>	Boolean telling if the scale coefficient should be read from persistent memory.

9.2.2.11 easy_save_to_persistent_memory_with_params()

```
void LoadCellController::easy_save_to_persistent_memory_with_params (
    byte loadcell_num,
    bool save_offset,
    bool save_scale ) [protected]
```

Easy function managing the saving of calibration parameters to persistent memory.

This function is managing the possibilities that come with the saving of the tare offset and the scale coefficient of a [LoadCell](#). It is meant to be used inside the member function [easy_start_with_params\(\)](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>save_offset</i>	Boolean telling if the offset should be saved to persistent memory.
<i>save_scale</i>	Boolean telling if the scale coefficient should be saved to persistent memory.

9.2.2.12 easy_start_with_params()

```
void LoadCellController::easy_start_with_params (
    byte loadcell_num,
    byte dout,
    byte pd_sck,
    bool calibrate_offset = false,
    bool calibrate_scale = false,
    bool read_offset_persistent_memory = false,
    bool read_scale_persistent_memory = false,
    bool save_offset_persistent_memory = false,
    bool save_scale_persistent_memory = false,
    float tare_offset = 0,
    float scale_coeff = 0,
    byte gain = 128 )
```

Easy function managing the start-up of a [LoadCell](#) with parameters as input.

This function, with only a few parameters as input, controls everything related to the start-up of a [LoadCell](#). The calibration of the offset and the scale coefficient and their saving/reading to/from the persistent memory. All possible conflicts between parameters are handled by the member function [easy_handle_exceptions\(\)](#).

This member function was created to simplify the start-up of a [LoadCell](#), especially for non-programmers. No additional functionalities that are not accessible from the other member functions are implemented (not labeled `easy_...`).

All the following “easy” member functions are used inside [easy_start_with_params\(\)](#) to dispatch the handling of all start-up scenarios:

- [easy_calibration_with_params\(\)](#)
- [easy_save_to_persistent_memory_with_params\(\)](#)
- [easy_read_from_persistent_memory_with_params\(\)](#)
- [easy_handle_exceptions\(\)](#)

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>dout</i>	Digital pin (or analog) connected to the DOUT output pin of the HX711.
<i>pd_sck</i>	Digital pin (or analog) connected to the SCK output of the HX711.
<i>calibrate_offset</i>	Boolean telling if the offset should be calibrated. Default is false.
<i>calibrate_scale</i>	Boolean telling if the scale coefficient should be calibrated. Default is false.
<i>read_offset_persistent_memory</i>	Boolean telling if the offset should be read from persistent memory. Default is false.
<i>read_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be read from persistent memory. Default is false.
<i>save_offset_persistent_memory</i>	Boolean telling if the offset should be saved to persistent memory. Default is false.
<i>save_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be saved to persistent memory. Default is false.
<i>tare_offset</i>	Value of the tare offset. Default is 0.
<i>scale_coeff</i>	Value of the scale coefficient. Default is 0.
<i>gain</i>	Gain of the HX711. Default is 128.

The tare_offset default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

The scale_coeff default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

9.2.2.13 easy_start_with_questions()

```
void LoadCellController::easy_start_with_questions (
    byte loadcell_num,
    byte dout,
    byte pd_sck,
    byte gain = 128 ) [protected]
```

Easy function for start with questions through the serial monitor.

This function is managing the possibilities that come with the start-up of a [LoadCell](#) with questions that are asked through the serial monitor of the Arduino.

Todo This could be done in the future to simplify the start-up of a [LoadCell](#) especially for non-programmers.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>dout</i>	Pin connected to the DOUT output of the HX711.
<i>pd_sck</i>	Pin connected to the SCK output of the HX711.
<i>gain</i>	Gain of the HX711. Default is 128

The `gain` argument can take the following values: 128, 64 and 32. For a gain of 128 and 64, channel A of the HX711 should be used. For a gain of 32, channel B should be used.

9.2.2.14 `get_offset_eeprom_address()`

```
int LoadCellController::get_offset_eeprom_address (
    byte loadcell_num )
```

Get the EEPROM address for the offset of a [LoadCell](#).

This function returns the EEPROM address for the offset value for a given [LoadCell](#). The EEPROM addresses are based on the loadcell number.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

EEPROM address for the offset value.

Note

The EEPROM addresses are spaced along the EEPROM to prevent overlap between neighboring values.

9.2.2.15 `get_offset_file_name()`

```
String LoadCellController::get_offset_file_name (
    byte loadcell_num )
```

Get the SPIFFS file name for the offset of a [LoadCell](#).

This function returns the offset file name for a given [LoadCell](#). The offset file name is based on the loadcell number.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

File name for the scale coefficient.

Note

Offset file name are formatted this way: `"/offset{loadcell number}.txt"`. For example, the offset file name for [LoadCell](#) #1 is `"/offset1.txt"`

9.2.2.16 get_scale_coeff_eeprom_address()

```
int LoadCellController::get_scale_coeff_eeprom_address (
    byte loadcell_num )
```

Get the EEPROM address for the scale coefficient of a [LoadCell](#).

This function returns the EEPROM address for the scale coefficient value for a given [LoadCell](#). The EEPROM addresses are based on the loadcell number.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

EEPROM address for the scale coefficient.

Note

The EEPROM addresses are spaced along the EEPROM to prevent overlap between neighboring values.

9.2.2.17 get_scale_coeff_file_name()

```
String LoadCellController::get_scale_coeff_file_name (
    byte loadcell_num )
```

Get the SPIFFS file name for the scale coefficient of a [LoadCell](#).

This function returns the scale coefficient file name for a given [LoadCell](#). The file name is based on the loadcell number.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

File name for the scale coefficient.

Note

Offset file name are formatted this way: "/offset{loadcell number}.txt". For example, the offset file name for [LoadCell](#) #1 is "/offset1.txt"

9.2.2.18 is_loadcell_num_in_range()

```
bool LoadCellController::is_loadcell_num_in_range (
    byte loadcell_num )
```

Verify if a [LoadCell](#) number is in the range of the [LoadCellController](#).

This function returns boolean telling whether a number of a [LoadCell](#) is in the range of the possible number of [LoadCell](#) for a [LoadCellController](#) according to his member variable `n_loadcell`. This function is useful inside all member functions that `loadcell_num` as parameters to suspend the code execution if necessary.

Returns

Boolean telling if a [LoadCell](#) number is coherent for the [LoadCellController](#).

9.2.2.19 number_of_loadcells()

```
byte LoadCellController::number_of_loadcells ( )
```

Get the number of [LoadCell](#) of the [LoadCellController](#).

This function returns the number of [LoadCell](#) that were added to the [LoadCellController](#) with the `add_loadcell()` member function since the `n_loadcell` member variable is kept protected.

Returns

Number of [LoadCell](#).

9.2.2.20 read_offset_from_persistent_memory()

```
long LoadCellController::read_offset_from_persistent_memory (
    byte loadcell_num )
```

Read the offset of a given [LoadCell](#) from persistent memory.

This function reads the offset of a [LoadCell](#) from persistent memory, which was added to the controller with the member function `add_loadcell()`. For a ATmega328p board, the EEPROM address is obtained with the member function `get_offset_eeprom_adress()`. The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function `get_offset_file_name()`. The same way, everything is hidden from the user.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

The offset of the given [LoadCell](#) obtained from persistent memory.

9.2.2.21 read_scale_coeff_from_persistent_memory()

```
float LoadCellController::read_scale_coeff_from_persistent_memory (
    byte loadcell_num )
```

Read the scale coefficient of a given [LoadCell](#) to persistent memory.

This function reads the offset of a [LoadCell](#) from persistent memory, which was added to the controller with the member function [add_loadcell\(\)](#). For a ATmega328p board, the EEPROM address is obtained with the member function [get_offset_eeprom_adress\(\)](#). The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function [get_offset_file_name\(\)](#). The same way, everything is hidden from the user.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

The scale coefficient of the given [LoadCell](#) obtained from persistent memory.

9.2.2.22 `save_offset_to_persistent_memory()`

```
void LoadCellController::save_offset_to_persistent_memory (
    byte loadcell_num )
```

Save the offset of a given [LoadCell](#) to persistent memory.

This function saves the offset member variable of a [LoadCell](#), which was added to the controller with the member function [add_loadcell\(\)](#). For a ATmega328p board, the EEPROM address is obtained with the member function [get_offset_eeprom_adress\(\)](#). The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function [get_offset_file_name\(\)](#). The same way, everything is hidden from the user.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

9.2.2.23 `save_scale_coeff_to_persistent_memory()`

```
void LoadCellController::save_scale_coeff_to_persistent_memory (
    byte loadcell_num )
```

Save the scale coefficient of a given [LoadCell](#) to persistent memory.

This function saves the scale coefficient member variable of a [LoadCell](#), which was added to the controller with the member function [add_loadcell\(\)](#). For a ATmega328p board, the EEPROM address is obtained with the member function [get_offset_eeprom_adress\(\)](#). The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function [get_offset_file_name\(\)](#). The same way, everything is hidden from the user.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

9.2.3 Member Data Documentation

9.2.3.1 loadcells

```
Loadcell * LoadCellController::loadcells [protected]
```

Array containing the pointer of the [LoadCell](#) controlled by the [LoadCellController](#).

The only moment this variable is modified is when a [LoadCell](#) is added with the [add_loadcell\(\)](#) member function. It goes up by one every time [add_loadcell\(\)](#) is called.

9.2.3.2 n_loadcell

```
int LoadCellController::n_loadcell = 0 [protected]
```

Member variable used to keep track of the number of LoadCells added.

The only moment this variable is modified is when a [LoadCell](#) is added with the [add_loadcell\(\)](#) member function. It goes up by one every time [add_loadcell\(\)](#) is called.

The documentation for this class was generated from the following files:

- LoadCellControllerlibrary /[LoadCellController.h](#)
- LoadCellControllerlibrary /[LoadCellController.cpp](#)

Chapter 10

File Documentation

10.1 LoadCellControllerlibrary /LoadCellController.h File Reference

```
#include <Arduino.h>
#include "HX711.h"
#include "LoadCell.h"
```

Classes

- class [LoadCellController](#)

This library allows to control and manage many load cells with a [LoadCellController](#).

10.2 LoadCellController.h

[Go to the documentation of this file.](#)

```
00001
00079 #ifndef LoadCellController_h
00080 #define LoadCellController_h
00081 #include <Arduino.h>
00082 #include "HX711.h"
00083 #include "LoadCell.h"
00084
00085 // Verify if the board is based on a ESP32 chip, like FireBeetle
00086 #if defined(ARDUINO_ARCH_ESP32)
00087 #include "FS.h"
00088 #include "SPIFFS.h"
00089 // Verify if the board is based on a Atmega328P chip, like Arduino Uno
00090 #elif defined(__AVR_ATmega328P__)
00091 #include <EEPROM.h>
00092 #endif
00093
00094
00095 class LoadCellController {
00096 protected:
00105     LoadCell* loadcells[10];
00106
00107
00116     int n_loadcell = 0;
00117
00132     void easy_calibration_with_params(
00133                                     byte loadcell_num,
00134                                     bool cal_tare,
00135                                     bool cal_scale
00136                                     );
00137
00138
```

```

00150 void easy_save_to_persistent_memory_with_params(
00151     byte loadcell_num,
00152     bool save_offset,
00153     bool save_scale
00154 );
00155
00156 void easy_read_from_persistent_memory_with_params(
00169     byte loadcell_num,
00170     bool read_offset,
00171     bool read_scale
00172 );
00173
00174 void easy_start_with_questions(
00191     byte loadcell_num,
00192     byte dout,
00193     byte pd_sck,
00194     byte gain=128
00195 );
00196
00197 void easy_handle_exceptions(
00222     byte loadcell_num,
00223     bool calibrate_offset=false,
00224     bool calibrate_scale=false,
00225     bool read_offset_persistent_memory=false,
00226     bool read_scale_persistent_memory=false,
00227     bool save_offset_persistent_memory=false,
00228     bool save_scale_persistent_memory=false,
00229     float tare_offset=0,
00230     float scale_coeff=0
00231 );
00232
00233 public:
00234 LoadCellController();
00235
00236 void add_loadcell(LoadCell &loadcell);
00237
00238 void easy_start_with_params(
00291     byte loadcell_num,
00292     byte dout,
00293     byte pd_sck,
00294     bool calibrate_offset=false,
00295     bool calibrate_scale=false,
00296     bool read_offset_persistent_memory=false,
00297     bool read_scale_persistent_memory=false,
00298     bool save_offset_persistent_memory=false,
00299     bool save_scale_persistent_memory=false,
00300     float tare_offset=0,
00301     float scale_coeff=0,
00302     byte gain=128
00303 );
00304
00305 void save_offset_to_persistent_memory(byte loadcell_num);
00318
00319 void save_scale_coeff_to_persistent_memory(byte loadcell_num);
00332
00333 long read_offset_from_persistent_memory(byte loadcell_num);
00347
00348 float read_scale_coeff_from_persistent_memory(byte loadcell_num);
00362
00363 void calibrate_both_params(byte loadcell_num);
00373
00374 void calibrate_tare_offset(byte loadcell_num);
00384
00385 void calibrate_scale_coeff(byte loadcell_num);
00395
00396 float determine_offset(byte loadcell_num);
00410
00411 float determine_scale_coeff(byte loadcell_num);
00439
00440 float calculate_scale_coeff(
00442     byte loadcell_num,
00443     float output,
00444     float mass
00445 );
00446
00447 int get_offset_eeprom_address(byte loadcell_num);
00462
00463 int get_scale_coeff_eeprom_address(byte loadcell_num);
00478
00479
00480

```

```
00494     String get_offset_file_name(byte loadcell_num);
00495
00496     String get_scale_coeff_file_name(byte loadcell_num);
00510
00511     float get_offset(byte loadcell_num);
00531
00532     void set_offset(byte loadcell_num, float offset);
00543
00544     float get_scale(byte loadcell_num);
00555
00556     void set_scale(byte loadcell_num, float scale);
00567
00568     void set_tare_n_readings(byte loadcell_num, int n_readings);
00569
00579     void set_tare_n_readings(byte loadcell_num);
00580
00581     byte get_tare_n_readings(byte loadcell_num);
00591
00592     void set_scale_coeff_n_readings(byte loadcell_num, int n_readings);
00593
00603     byte get_scale_coeff_n_readings(byte loadcell_num);
00604
00605     void set_weight_n_readings(byte loadcell_num, int n_readings);
00615
00616     byte get_weight_n_readings(byte loadcell_num);
00617
00627     void set_all_loadcells_tare_n_readings(int n_readings);
00628
00629     byte get_weight_n_readings(byte loadcell_num);
00639
00640     void set_all_loadcells_tare_n_readings(int n_readings);
00641
00650     void set_all_loadcells_scale_coeff_n_readings(int n_readings);
00651
00652     void set_all_loadcells_weight_n_readings(int n_readings);
00661
00662     long read_raw_average(byte loadcell_num);
00663
00672     long read_tare_average(byte loadcell_num);
00673
00674     long read_scale_coeff_average(byte loadcell_num);
00685
00686     double get_raw_value(byte loadcell_num);
00687
00698     float get_weight(byte loadcell_num);
00699
00710     void tare(byte loadcell_num);
00711
00712     void begin(
00723         byte loadcell_num,
00724         byte dout,
00725         byte pd_sck,
00737         byte gain = 128
00738     );
00739
00748     bool is_ready(byte loadcell_num);
00749
00750     void wait_ready(
00765         byte loadcell_num,
00766         unsigned long delay_ms = 0
00767     );
00768
00769     bool wait_ready_retry(
00770         byte loadcell_num,
00771         int retries = 3,
00772         unsigned long delay_ms = 0
00773     );
00786
00787     bool wait_ready_timeout(
```

```

00843             byte loadcell_num,
00844             unsigned long timeout = 1000,
00845             unsigned long delay_ms = 0
00846             );
00847
00848
00857     void power_up(byte loadcell_num);
00858
00859
00870     void power_down(byte loadcell_num);
00871 // End of LoadCell functions for LoadCellController group
00872
00873
00874     byte number_of_loadcells();
00885
00886
00887
00898     bool is_loadcell_num_in_range(byte loadcell_num);
00899
00911     void clear_eeprom(int start=0, int end=1024);
00912 };
00913 #endif

```

10.3 LoadCellLibrary/LoadCell.h File Reference

```

#include <Arduino.h>
#include "HX711.h"

```

Classes

- class [LoadCell](#)

This library extends the functionality of the HX711 to make it compatible with the [LoadCellController](#) library.

10.4 LoadCell.h

[Go to the documentation of this file.](#)

```

00001
00037 #ifndef LoadCell_h
00038 #define LoadCell_h
00039 #include <Arduino.h>
00040 #include "HX711.h"
00041 class LoadCell : public HX711 {
00042
00043 protected:
00048     byte tare_n_readings = 50;
00049
00050
00055     byte scale_coeff_n_readings = 50;
00056
00057
00062     byte weight_n_readings = 5;
00063 public:
00067     LoadCell();
00068
00069
00078     long read_raw_average();
00079
00080
00089     long read_tare_average();
00090
00091
00100     long read_scale_coeff_average();
00101
00102
00111     double get_raw_value();
00112
00113
00123     float get_weight();
00124

```

```
00125
00133     void tare();
00134
00135
00146     void set_tare_n_readings(int tare_n_readings);
00147
00148
00156     int get_tare_n_readings();
00157
00158
00169     void set_scale_coeff_n_readings(int scale_coeff_n_readings);
00170
00171
00179     int get_scale_coeff_n_readings();
00180
00181
00192     void set_weight_n_readings(int weight_n_readings);
00193
00194
00202     int get_weight_n_readings();
00203 };
00204 #endif
```

